

# FAST NATIONAL UNIVERSITY

CHINIOT-FAISALABAD CAMPUS

*Department of Computer Science*

---

## PROJECT ASSIGNMENT

# COMPILER CONSTRUCTION

*Build a Mini C Language Compiler*

Demonstrating All Six Compiler Phases with Visible Output

|                     |                       |
|---------------------|-----------------------|
| Course              | Compiler Construction |
| Project Type        | Individual Project    |
| Total Marks         | 100 Marks             |
| Submission Deadline | 10/05/2026            |

# 1. Project Overview

---

This project requires each student to design and implement a Mini C Language Compiler . The compiler must process a subset of the C programming language and produce visible, well-formatted output for every phase of compilation.

The goal is to reinforce theoretical concepts taught in the Compiler Construction course by building a practical, working system. Students must not simply simulate outputs — every phase must be genuinely implemented and derived from the previous phase's output.

## 1.1 Supported Subset of C Language

Your compiler must correctly handle the following constructs of C:

- Data type: int only
- Variable declarations: `int x;` and `int x = 5;`
- Arithmetic expressions: `+`, `-`, `*`, `/` (with precedence and parentheses)
- Assignment statements: `x = expr;`
- Relational operators: `<`, `>`, `<=`, `>=`, `==`, `!=`
- if and if-else statements (with nested support)
- while loop
- for loop
- Function declarations and calls (single return type: int)
- return statement
- Single-line comments: `// comment`

## 2. Allowed Tools & Technologies

---

Students may use the following tools for implementation:

- Implementation Language: C
- Lexer Generator: Lex or Flex (for Phase 1 – Lexical Analysis)
- Parser Generator: Yacc or Bison (for Phase 2 – Syntax Analysis)
- Build System: Makefile or manual compilation steps
- IDE / Editor: Any (VS Code, CLion, Vim, etc.)
- OS: Linux/Mac ; Windows with MinGW/WSL

**Note:** All remaining phases (Semantic Analysis, Intermediate Code, Optimization, Code Generation) must be implemented manually in C. No additional generators or libraries are allowed for these phases.

### 3. Compiler Phases – Requirements & Deliverables

Your compiler must implement all six standard phases. For each phase, a clear, formatted output must be printed to the console AND included as a screenshot in your project report.

| Phase 1: Lexical Analysis (Tokenization) |                                                                                                          |                                                                                                                                            |                |       |
|------------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|----------------|-------|
| #                                        | Description                                                                                              | Required Output / Deliverable                                                                                                              | Sample Input   | Marks |
| 1                                        | Read the source .c file character-by-character using Flex/Lex. Identify and classify all lexical tokens. | Print a token stream table: each row shows Token Value, Token Type (KEYWORD, IDENTIFIER, OPERATOR, LITERAL, PUNCTUATION), and Line Number. | int x = 5 + 3; | 10    |

#### Example Output – Phase 1:

| Token Value | Token Type      | Line No. |
|-------------|-----------------|----------|
| int         | KEYWORD         | 1        |
| x           | IDENTIFIER      | 1        |
| =           | OPERATOR        | 1        |
| 5           | INTEGER_LITERAL | 1        |
| +           | OPERATOR        | 1        |
| 3           | INTEGER_LITERAL | 1        |
| ;           | PUNCTUATION     | 1        |

| Phase 2: Syntax Analysis (Parsing & Parse Tree) |                                                                                                                                              |                                                                                                                               |                |       |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|----------------|-------|
| #                                               | Description                                                                                                                                  | Required Output / Deliverable                                                                                                 | Sample Input   | Marks |
| 2                                               | Use Bison/Yacc to define the grammar of your Mini-C subset. Parse the token stream and detect syntax errors. Build and display a Parse Tree. | Print a visual parse tree (indented text or ASCII-art). On syntax error, print: 'Syntax Error at line X: unexpected token Y'. | int x = 5 + 3; | 15    |

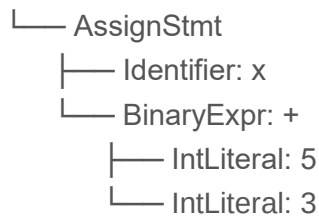
#### Example Output – Phase 2:

Program

```

└─ FunctionDecl: main
    └─ Body

```



### Phase 3: Semantic Analysis

| # | Description                                                                                                                                          | Required Output / Deliverable                                                                                                            | Sample Input   | Marks |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|----------------|-------|
| 3 | Traverse the parse tree / AST and enforce semantic rules: type checking, undeclared variable detection, duplicate declaration, and scope validation. | Print a Symbol Table (name, type, scope, line declared). Print any semantic errors found. Print 'Semantic Analysis Passed' if no errors. | int x = 5 + 3; | 15    |

#### Example Output – Phase 3:

| Variable | Type | Scope  | Line Decl. | Status |
|----------|------|--------|------------|--------|
| x        | int  | global | 1          | OK     |
| result   | int  | main() | 3          | OK     |

### Phase 4: Intermediate Code Generation (Three-Address Code)

| # | Description                                                                                                                                | Required Output / Deliverable                                                                                               | Sample Input   | Marks |
|---|--------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|----------------|-------|
| 4 | Generate Three-Address Code (TAC) from the AST. Use temporary variables (t1, t2, ...) to break complex expressions into simple operations. | Print the TAC instructions sequentially. Each line: 'tN = operand1 op operand2' or 'if cond goto LN' or 'param X / call F'. | int x = 5 + 3; | 15    |

#### Example Output – Phase 4 (for: int x = 5 + 3 \* 2;):

t1 = 3 \* 2

t2 = 5 + t1

x = t2

### Phase 5: Code Optimization

| # | Description                                | Required Output / Deliverable                                     | Sample Input   | Marks |
|---|--------------------------------------------|-------------------------------------------------------------------|----------------|-------|
| 5 | Apply at least two optimization techniques | Print the original TAC alongside the optimized TAC. Clearly label | int x = 5 + 3; | 10    |

|  |                                                                                                                                          |                                                |  |  |
|--|------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|--|--|
|  | on the TAC: (1) Constant Folding – evaluate constant expressions at compile time. (2) Dead Code Elimination – remove unused assignments. | which optimization was applied to each change. |  |  |
|--|------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|--|--|

**Example Output – Phase 5:**

Before Optimization:  $t1 = 3 * 2 \rightarrow$  After (Constant Folding):  $t1 = 6$

Before Optimization:  $t2 = 5 + t1 \rightarrow$  After (Constant Folding):  $x = 11$

| Phase 6: Target Code Generation (Assembly / Pseudo-Code) |                                                                                                                                                 |                                                                                                                                           |                          |       |
|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|-------|
| #                                                        | Description                                                                                                                                     | Required Output / Deliverable                                                                                                             | Sample Input             | Marks |
| 6                                                        | Translate the optimized TAC into x86-style assembly language or a well-defined pseudo-assembly. Map variables to registers or memory locations. | Print the generated assembly/pseudo-code. Include data section (variable declarations), text section (instructions), and return handling. | $\text{int } x = 5 + 3;$ | 15    |

**Example Output – Phase 6 (x86-style pseudo-assembly):**

section .data

x dd 0

section .text

MOV EAX, 11

MOV [x], EAX

RET

| Phase 7: Program Execution & Final Output |                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                       |                          |       |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|-------|
| #                                         | Description                                                                                                                                                                                                                                                                                                          | Required Output / Deliverable                                                                                                                                                                                         | Sample Input             | Marks |
| 7                                         | After all 6 phases, your compiler must EXECUTE the program and display the actual output — just like a real compiler. Implement a TAC Interpreter: walk through the optimized TAC, evaluate all expressions, execute control flow (if/else, loops), handle function calls, and print printf() results to the screen. | Given any valid Mini-C source file, print all phase outputs (1–6), then a separator line, then 'Program Output:' followed by the actual computed result. E.g. if input computes $2+3$ , the final output must show: 5 | $\text{int } x = 5 + 3;$ | 10    |

## 4. Submission Requirements

---

Each group must submit the following as a single ZIP file named: **GroupNo\_CC\_Project.zip**

| # | Item                                     | Status   |
|---|------------------------------------------|----------|
| 1 | Source code (well-commented C/C++ files) | Required |
| 2 | Makefile or build instructions           | Required |
| 3 | Sample input .c files used for testing   | Required |
| 4 | Screenshot outputs for each phase        | Required |
| 5 | Project report (PDF, min 15 pages)       | Required |
| 6 | README.txt with compilation & run steps  | Required |

### 4.1 Report Structure (Minimum 15 Pages)

1. Title Page – University, course, group members (name + roll no.)
2. Table of Contents
3. Introduction & Problem Statement
4. Grammar Specification (BNF / EBNF rules for your Mini-C subset)
5. Phase-by-Phase Design & Implementation (one section per phase)
6. Screenshots of output for each phase
7. Challenges faced and how they were resolved

### 4.2 Demo / Viva Guidelines

- Each Student must be present for the demo.
- Demonstrate live compilation of at least three test .c files: (a) a simple assignment, (b) an if-else block, (c) a function with a loop.
- Each phase output must be visible on-screen during the demo.
- Demos will be scheduled by the instructor after the submission deadline.

## 5. Tips for Success

---

- Start with Phase 1 (Lexer) first — all later phases depend on it.
- Test each phase independently before connecting them.

- Write clean, modular code — separate files for each phase are strongly recommended.
  - Use Flex/Bison documentation and course slides as your primary references.
  - Test with edge cases: nested if-else, function calls inside expressions, missing semicolons.
  - Keep regular backups of your code (use GitHub or similar).
  - Divide work early — one member can handle Phases 1-3, the other Phases 4-6, then merge and test together.
- 

***Good Luck! Build something real. Build something you are proud of.***

FAST National University – CFD Campus | Department of Computer Science